Introduction to Machine Learning in R

One dataset, one prediction, and the process that doesn't change

juni 2026

Contents

Scope . . . 1
The data . . . 2
Linear regression as a baseline . . . 2
We need more data! . . . 3
The process: split, fit, score . . . 5
The complexity ladder . . . 5
Feature engineering can rescue the simple model . . . 7
Complexity as a double-edged sword . . . 8
How much can we trust one split? . . . 8
Reading the black box . . . 9
The same workflow, in the real tool . . . 10
You have a model. Now what? . . . 12
Where to go next . . . 12

Scope

Machine learning sounds daunting, perhaps even sci-fi. It's really one **process**, and the model is a part you swap in and out:

> **split** the data → **fit** a model on one part → **score** it on the part it never saw.

That last step is the whole discipline. Everything else is detail.

In this tutorial we predict one thing: how much CO_2 a grass plant takes up, and we move from the simplest model (a straight line) to a model of higher complexity (a random forest), running the *same* process each time and using a *hold-out set* as an objective 'referee'. Mind you that an increase in complexity does *not* always pay off!

```
# install once if needed:
# install.packages(c("rpart", "randomForest"))
library(rpart)         # decision trees
library(randomForest)  # random forests
```

The data

`CO2` is built into R, so there's nothing to download. It records CO_2 uptake in a grass (*Echinochloa crus-galli*) grown in **Quebec** or **Mississippi**, **chilled** or not (treatment), measured across seven ambient CO_2 **concentrations**. We'll predict `uptake`.

```
# ?CO2
d <- as.data.frame(CO2)
d$Plant <- NULL          # an experiment label, not a predictor, we're dropping it (~feature selection)
head(d)
```

```
##     Type  Treatment conc uptake
## 1 Quebec nonchilled   95   16.0
## 2 Quebec nonchilled  175   30.4
## 3 Quebec nonchilled  250   34.8
## 4 Quebec nonchilled  350   37.2
## 5 Quebec nonchilled  500   35.3
## 6 Quebec nonchilled  675   39.2
```

```
summary(d)
```

```
##           Type         Treatment       conc          uptake
##  Quebec     :42   nonchilled:42   Min.   :  95   Min.   : 7.70
##  Mississippi:42   chilled   :42   1st Qu.: 175   1st Qu.:17.90
##                                   Median : 350   Median :28.30
##                                   Mean   : 435   Mean   :27.21
##                                   3rd Qu.: 675   3rd Qu.:37.12
##                                   Max.   :1000   Max.   :45.50
```

Linear regression as a baseline

The most familiar model you have is a straight line through a point cloud. Let's fit one to investigate whether plant uptakes rise with ambient CO_2 concentration.

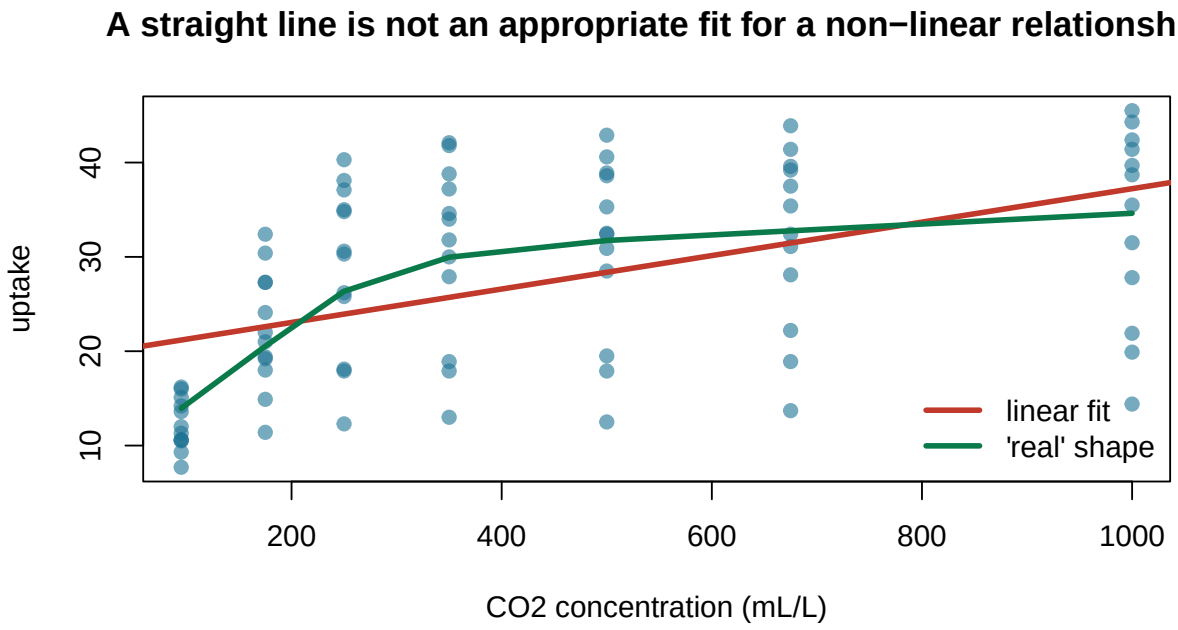
```
m1 <- lm(uptake ~ conc, data = d)

summary(m1)
```

```
##
## Call:
## lm(formula = uptake ~ conc, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -22.831  -7.729   1.483   7.748  16.394
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 19.500290   1.853080  10.523  < 2e-16 ***
## conc         0.017731   0.003529   5.024 2.91e-06 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.514 on 82 degrees of freedom
## Multiple R-squared:  0.2354, Adjusted R-squared:  0.2261
## F-statistic: 25.25 on 1 and 82 DF,  p-value: 2.906e-06
```

```r
plot(uptake ~ conc, data = d, pch = 19, col = "#1C729399",
     xlab = "CO2 concentration (mL/L)", ylab = "uptake",
     main = "A straight line is not an appropriate fit for a non-linear relationship")
abline(m1, col = "#C0392B", lwd = 3)                  # the linear fit
lines(lowess(d$conc, d$uptake), col = "#0B7A4B", lwd = 3)  # the real shape
legend("bottomright", c("linear fit", "'real' shape"),
       col = c("#C0392B", "#0B7A4B"), lwd = 3, bty = "n")
```



CO_2 uptake climbs steeply, then **saturates** towards an asymptote. The green curve tracks it while the red line doesn't.

We need more data!

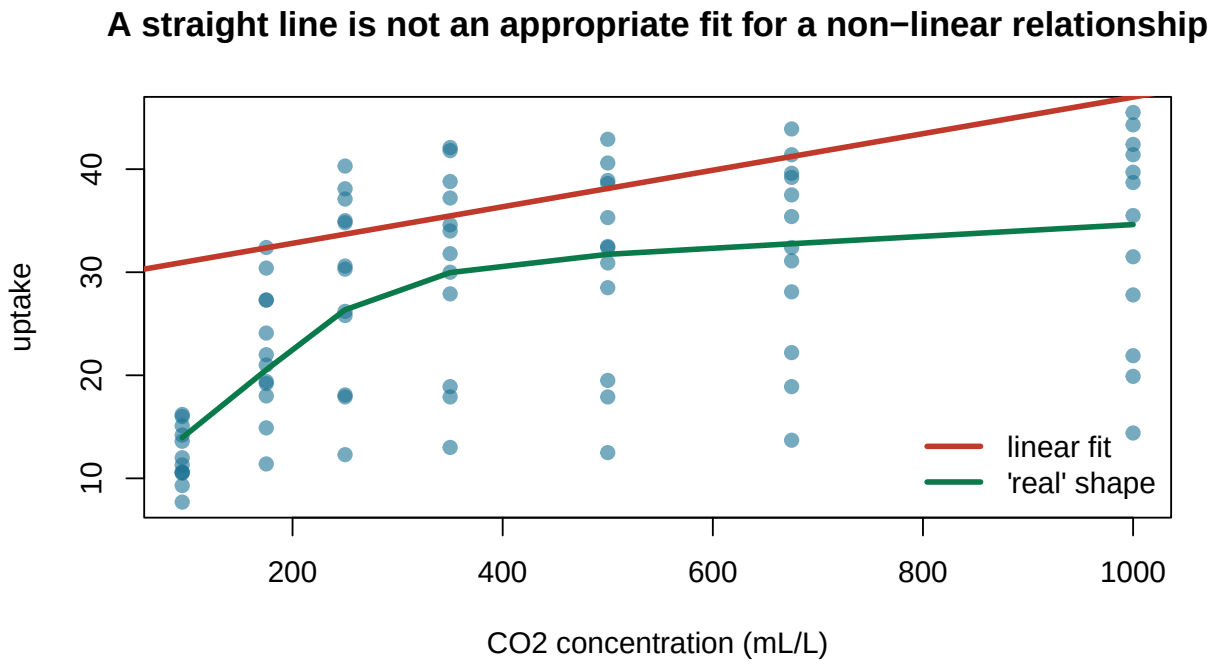
A frustration you'll hear exclaimed most often is the need for more data. Indeed, adding more predictors may help improve the model's performance. But does it also work for our question?

```r
m2 <- lm(uptake ~ conc + Type + Treatment, data = d)
summary(m2)
```

```
##
## Call:
```

```
## lm(formula = uptake ~ conc + Type + Treatment, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -17.344  -2.826   1.070   4.137  11.267
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)       29.259814   1.538928  19.013  < 2e-16 ***
## conc               0.017731   0.002297   7.719 2.87e-11 ***
## TypeMississippi  -12.659524   1.351439  -9.367 1.67e-14 ***
## Treatmentchilled  -6.859524   1.351439  -5.076 2.46e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.193 on 80 degrees of freedom
## Multiple R-squared:  0.6839, Adjusted R-squared:  0.6721
## F-statistic:  57.7 on 3 and 80 DF,  p-value: < 2.2e-16
```

```r
plot(uptake ~ conc, data = d, pch = 19, col = "#1C729399",
     xlab = "CO2 concentration (mL/L)", ylab = "uptake",
     main = "A straight line is not an appropriate fit for a non-linear relationship")
abline(m2, col = "#C0392B", lwd = 3)                  # the linear fit
lines(lowess(d$conc, d$uptake), col = "#0B7A4B", lwd = 3)  # the real shape
legend("bottomright", c("linear fit", "'real' shape"),
       col = c("#C0392B", "#0B7A4B"), lwd = 3, bty = "n")
```



There is an increase in adj. R^2 (22.6% -> 67.2%) - adding more data worked! Here, observe how the *chilled*

treated plants take up about 6.86 units *less* than their *non-chilled* counterparts. However, looking at the data, we can agree it's not *quite* there yet.

Before moving to Machine Learning, it's worth nothing that there is a distinct shift in focus between *inference* and *prediction.* Traditional statistical models (like classic linear regression) are often used to *infer* relationships and understand underlying mechanisms in data. In contrast, machine learning models prioritize *pattern recognition* to maximize accuracy when predicting *unseen* data.

The process: split, fit, score

So far we've only fit models and looked at them: the line on the plot, the coefficients and p-values in the summary. We also rated the fit with the R-squared, which shows how much of the variance in the data the model explains. But notice what that number really measures: how well the model predicts its own data -> the data it was trained on.

That's the catch. Whenever we're predicting (the supervised side of machine learning) we always split our data into a *training* and a *testing* set, and judge the model only on the test set. Otherwise we can never tell how well it will predict data it hasn't seen.

```
set.seed(42)                                    # reproducible split
n <- nrow(d)
train_idx <- sample(seq_len(n), floor(0.60 * n))  # 50 train, 34 test
train <- d[train_idx, ]
test  <- d[-train_idx, ]

# a quick test-set R^2 helper
r2 <- function(pred, actual) cor(pred, actual)^2
```

Fit on `train`. Score on `test`, which has been sitting in a 'sealed envelope'. Every model below runs through this same loop.

The complexity ladder

We run the *identical* process with models of rising complexity and see how the test score changes.

```
# rung 1 - a line, concentration only
m_uni <- lm(uptake ~ conc, data = train)
round(r2(predict(m_uni, test), test$uptake), 2)
```

```
## [1] 0.16
```

```
# rung 2 - a line, all three predictors (the jump here is better INPUTS, not a better model)
m_lin <- lm(uptake ~ conc + Type + Treatment, data = train)
round(r2(predict(m_lin, test), test$uptake), 2)
```

```
## [1] 0.58
```

The next two models don't work like `lm` at all. A linear model fits **one global equation**: it assumes a shape (a straight line) and finds the best one. The ML models below do not a priori assume a shape.

A **regression tree** asks a chain of yes/no questions about the predictors ("is concentration above 300? is it Quebec?"). Each question splits the data into smaller groups, and the tree predicts the average uptake

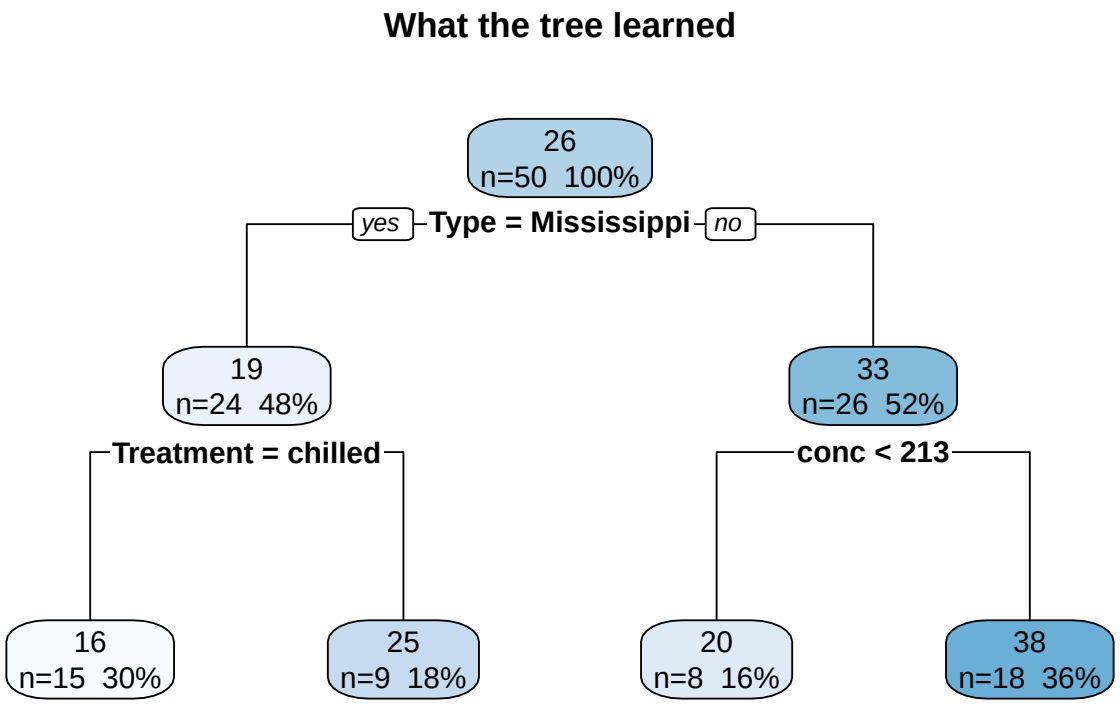
inside each final group. It's a flowchart it learns from the data, and because it can keep splitting, it follows the curve in steps.

The algorithm chooses its questions 'greedily'. At each step the tree tries every predictor and every possible cut-point, and keeps the one split that makes the two resulting groups most uniform in uptake. Then it repeats inside each group. The first split here landed on origin because splitting Quebec from Mississippi tightened the spread of uptake more than any split on concentration or chilling would have. It stops when more splitting barely helps — otherwise it would carve the data down to one leaf per branch (overfitting!).

```
# rung 3 - a regression tree (it can bend)
m_tree <- rpart(uptake ~ conc + Type + Treatment, data = train)
round(r2(predict(m_tree, test), test$uptake), 2)
```

```
## [1] 0.72
```

```
library(rpart.plot)        # install.packages("rpart.plot") once
rpart.plot(m_tree, type = 2, extra = 101, box.palette = "Blues",
           main = "What the tree learned")
```



A **random forest** is hundreds of such trees, each grown on a random handful of rows and offered a random subset of predictors at each split, so each tree is unique. To predict, it asks them all and averages (all trees get to 'vote'). That averaging cancels the quirks and noise of any single tree, which is why a forest is steadier and usually more accurate.

```r
# rung 4 - a random forest (many varied trees, averaged)
set.seed(42)
m_rf <- randomForest(uptake ~ conc + Type + Treatment, data = train)
m_rf
```

```
##
## Call:
##  randomForest(formula = uptake ~ conc + Type + Treatment, data = train)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 1
##
##           Mean of squared residuals: 31.62459
##                     % Var explained: 73.52
```

```r
round(r2(predict(m_rf, test), test$uptake), 2)
```

```
## [1] 0.89
```

```r
# devtools::install_github("araastat/reprtree")
# library(reprtree)
#
# # Find and plot the most representative tree in the ensemble
# rep_tree <- ReprTree(m_rf, train)
# plot(rep_tree)
```

Step back and look at the shape. Roughly **0.16 → 0.58 → 0.72 → 0.89**. Complexity climbed and, *here*, predictive power climbed with it. The relationship is genuinely curved, and the flexible models can follow a curve the line had trouble modelling.

However, the only reason we could verify the predictive power of these models, is because we used the *test set* we held out from training the model. Here, we used the predictive power of the model (the R-squared, if you will) based on the *test set* to rank the 4 models we created. But a higher complexity does not always yield higher predictive power!

Feature engineering can rescue the simple model

The line got stuck at 0.58 because it assumed a *straight* effect of concentration. However, from Biology we know that life does not really deal with infinites, but reaches a saturated state. In other words, we can engineer one of our features to account for this by computing the *log of concentration*.

```r
m_log <- lm(uptake ~ log(conc) + Type + Treatment, data = train)
round(r2(predict(m_log, test), test$uptake), 2)
```

```
## [1] 0.73
```

That lifts the line to about **0.73**, level with the tree, while staying a clean, readable equation. This is *feature engineering*, and it's the "which feature should I build?" question answering itself with a bit of domain knowledge. The forest reached the curve on its own, without being told its shape. That's part of what complexity buys you, as ML models are great at finding structure and patterns you'd otherwise have to know to encode.

Complexity as a double-edged sword

Above, complexity won. SO fancier is always better, ... right? Let's introduce some noise to our dataset. We'll generate 30 fake features of pure random noise.

```
set.seed(1)
k <- 30
noise <- as.data.frame(matrix(rnorm(n * k), ncol = k))
names(noise) <- paste0("noise", seq_len(k))
dn  <- cbind(d, noise)
trn <- dn[train_idx, ]
ten <- dn[-train_idx, ]

f_over <- as.formula(paste("uptake ~ conc + Type + Treatment +",
                           paste(names(noise), collapse = " + ")))

m_over <- lm(f_over, data = trn)

round(c(train_R2 = summary(m_over)$r.squared,
        test_R2  = r2(predict(m_over, ten), ten$uptake)), 2)
```

```
## train_R2  test_R2
##     0.93     0.25
```

The **training** R^2 jumps to about 0.93 = the model fit the noise. The **test** R^2 collapses to about 0.25. This is classical *overfitting*.

```
# same 30 noise columns - does the forest overfit too?
set.seed(42)
rf_over <- randomForest(f_over, data = trn)
c(forest_train = round(r2(predict(rf_over, trn), trn$uptake), 2),
  forest_test  = round(r2(predict(rf_over, ten), ten$uptake), 2))
```

```
## forest_train  forest_test
##         0.99         0.65
```

You've now seen both edges of the same knife. Complexity *helped* (the forest) and complexity *hurt* (the noise), and you cannot tell which from the training score. The held-out test is the only honest referee, in both directions. That's why it's the spine of everything here.

This phenomenon has a name: the **bias–variance tradeoff**. Too stiff and you underfit (the straight line, missing the curve); too flexible and you overfit (this). Every model lives somewhere on that tension. And one trap to watch for: **leakage**. Each plant in `CO2` was measured at all seven concentrations, so splitting *rows* can put one plant in both train and test, quietly inflating your score. The honest fix is to split along the unit you want to generalize to (here, by plant). See also this great interactive visualization of the bias-variance tradeoff: https://mlu-explain.github.io/bias-variance/.

How much can we trust one split?

Every score so far came from a *single* 60/40 split. That's a little like judging a die from one roll: a different split nudges the number. **Cross-validation** is the fix. In *5-fold* CV we cut the data into five parts, fit on four and score on the fifth, then rotate until each part has been the test set exactly once. Five honest scores instead of one.

```r
set.seed(42)
k <- 5
folds <- sample(rep(1:k, length.out = nrow(d)))   # assign each row to a fold
cv_r2 <- numeric(k)
for (i in seq_len(k)) {
  tr <- d[folds != i, ]; te <- d[folds == i, ]
  set.seed(42)
  m  <- randomForest(uptake ~ conc + Type + Treatment, data = tr)
  cv_r2[i] <- r2(predict(m, te), te$uptake)
}
round(cv_r2, 2)         # the five fold scores
```

```
## [1] 0.92 0.89 0.92 0.90 0.91
```

```r
round(mean(cv_r2), 2)   # the averaged estimate
```

```
## [1] 0.91
```

The five folds land close together and average about **0.91**, a sturdier figure than the single-split 0.89, because it no longer hinges on one lucky draw. The spread across folds is your rough sense of how much that number might wobble.

One ecological caveat, worth a sentence even if we don't chase it today: random folds assume the rows are *independent*, and **ours aren't**. Each plant was measured at all seven concentrations, so a plant lands in several folds at once and the model is never really tested on a *new* plant. That makes the estimate look steadier than it honestly is. The same problem haunts **spatial** and **temporal** data, where neighbouring sites or adjacent timepoints carry each other's information. The honest fix is the same idea as splitting by plant: hold out whole *groups* (a plant, a region, a time window) rather than random rows, which is **blocked** or **spatial cross-validation** (see the `blockCV` package). If you ever watch an AI hand spatially structured data to a plain random split, that's the mistake to catch.

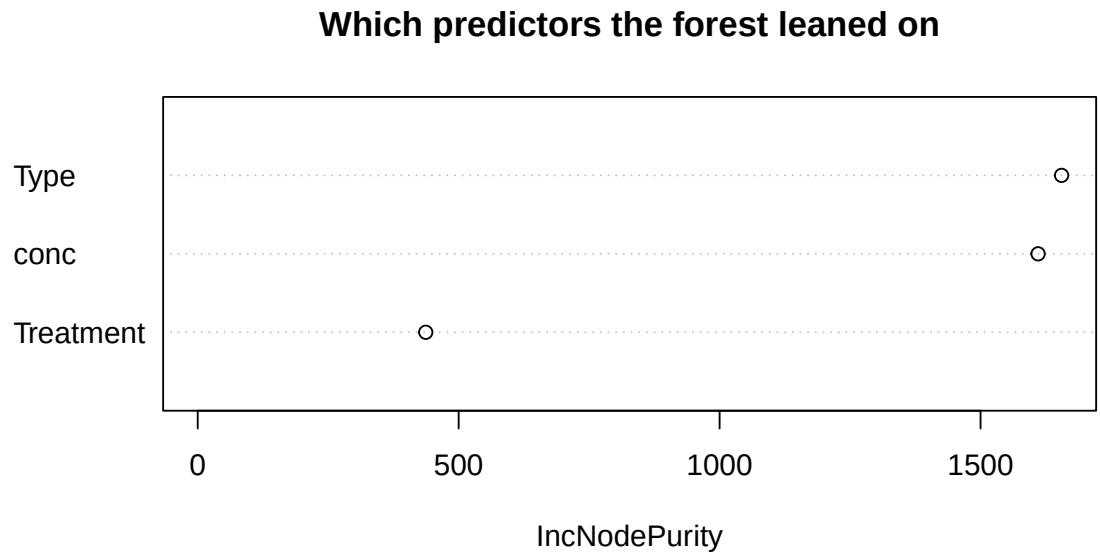
Reading the black box

The linear model was not as powerful as the forest, but it was much easier to interpret. How do we proceed with the interpretation of a ML model? Notice that an LM has an equation that describes the model, but we did not observe this for the random forest! That's the *black-box problem*. A model that predicts well but you can't really say why is hard to defend. Fortunately, there is a way to solve this!

```r
importance(m_rf)
```

```
##           IncNodePurity
## conc          1610.2554
## Type          1655.2910
## Treatment      436.9797
```

```r
varImpPlot(m_rf, main = "Which predictors the forest leaned on")
```



You can ask even a black box which inputs it used: concentration and origin carry most of the forest's accuracy, treatment much less. That answers the "which features?" question and tells you what you could drop. Two cautions: this is **not** causation (it's what the model used, not what drives the biology), and if you genuinely need to *explain* the model, prefer one you can read (recall the log-linear model scored 0.73 and is a clean equation). Sometimes the small accuracy cost is worth the transparency. It all depends on your research question (and on your data)!

The same workflow, in the real tool

We built everything by hand so the parts are visible. But different models are coded in different R packages, with each a different syntax to learn. One way to explore more models while decreasing this 'jargon burden' is by using the **tidymodels** package, which runs the identical process in one grammar.

```
# install.packages("tidymodels")
library(tidymodels)

set.seed(42)
split <- initial_split(CO2, prop = 0.70)        # the split, one line
train <- training(split)
test <- testing(split)

f <- uptake ~ conc + Type + Treatment

lin_fit <- linear_reg() |> fit(f, data = train)           # the line

rf_fit  <- rand_forest(mode = "regression") |>            # swap one function...
  set_engine("randomForest") |>                           # ...same engine as above
  fit(f, data = train)

augment(rf_fit, test) |>                                   # predict on held-out data, then
  metrics(truth = uptake, estimate = .pred)                # honest scores (rsq ~ 0.89)
```

```
# Want the tree? decision_tree(mode = "regression"). Same loop, one word.
```

Below is the real payoff of learning the workflow instead of one model's syntax: the model becomes a parameter. Every line below drops into the exact same split > fit > score loop you just ran. `linear_reg()` you've met, `rand_forest()` won today, and the rest are yours to try. The held-out test set tells you which one earned its place. Mind you that, in some cases, additional packages will be required (as the machinery from existing packages is used, but encoded into the tidymodels grammar).

```
# The model is the ONLY line that changes. The same
#   fit(f, data = train) |> augment(test) |> metrics(truth = uptake, ...)
# wraps every one of these. Pick the question first, then try a few.

linear_reg()                                        # a straight line (where we began)
linear_reg(penalty = 0.1) |> set_engine("glmnet")   # regularized: shrinks/drops weak features for yo
nearest_neighbor(neighbors = 5, mode = "regression")  # KNN: predict from the most similar plants
decision_tree(mode = "regression")                  # one readable tree
rand_forest(mode = "regression")                     # many trees averaged (today's winner)
boost_tree(mode = "regression")                      # gradient boosting: often strongest on tabular
svm_rbf(mode = "regression")                          # support vector machine
mlp(mode = "regression")                              # a small neural network
# ...and more - mars() for splines, bag_tree(), etc. Full list at tidymodels.org/find/parsnip
```

> One honest caveat you can now catch: `initial_split()` won't select the same rows as the hand-written `sample()` above, even with the same seed because the underlying machinery differs. The score barely moves here, but on your own data the exact split can change the answer. You know to check that now, because you saw the manual version first.

Try one yourself

Reading a menu is one thing; the point is to *try* one. Here's a tiny template that uses the `train` and `test` you already made: drop in any model from the list above, run it, and let the test set tell you whether it earned its place. The predicted-vs-actual plot at the end works for *any* regression model, so recycle it freely.

```
library(tidymodels)
library(ggplot2)

# 1. PICK A MODEL - swap this one line with anything from the menu above
model_spec <- rand_forest(mode = "regression") |> set_engine("randomForest")
# model_spec <- boost_tree(mode = "regression")  # for example

# 2. BUILD it on the training data
f   <- uptake ~ conc + Type + Treatment
fit <- model_spec |> fit(f, data = train)

# 3. VALIDATE on the held-out test set
scored <- augment(fit, new_data = test)              # adds a .pred column
scored |> metrics(truth = uptake, estimate = .pred)  # rmse, rsq, mae

# 4. LOOK at it: predicted vs actual (recycle for ANY regression model)
ggplot(scored, aes(uptake, .pred)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey50") +
```

```
  geom_point(size = 2, alpha = 0.8, colour = "#1C7293") +
  labs(x = "actual uptake", y = "predicted uptake",
       title = "Predicted vs actual on the test set",
       subtitle = "the closer to the dashed line, the better the prediction") +
  theme_minimal()
```

You have a model. Now what?

A model that scores well is the *start* of the conversation, not the end. A few questions you now know to ask:

- **Is it good enough?** "Good enough" isn't a universal number, it's relative to your question and a baseline. Does it beat just predicting the mean? Does it beat the simple model you'd have used anyway? Does it actually answer the biology? When extra complexity stops moving the *test* score, you've hit this data's ceiling.
- **Can I tune it?** Every model past the straight line has knobs, a tree's depth, a forest's number of trees, KNN's k. These are *hyperparameters*, and tuning them is its own little search, judged (of course) on held-out data. This tutorial should serve to get you curious and started. Once you're settling on a specific type of model, you can dive into its intricacies and requirements. More information on tuning with tidymodels: https://www.tmwr.org/tuning.

None of these is exotic, they're the next questions in the *same* flow. And they're exactly what to ask an AI assistant to set up, then check that it did. Knowing the flow is what lets you *direct* the tool instead of trusting it blindly.

Where to go next

To go further, see the companion resource sheet for datasets to practice on, free books (*Tidy Modeling with R*; *An Introduction to Statistical Learning*, whose Chapter 8 covers the trees and forests above), interactive overfitting demos, and how to open up black-box models. First move: drop a different dataset into this document and re-run the ladder. When it breaks, ask an AI to *explain* the error, not just fix it (or try for yourself :)).

```
## Built with:

## [1] "R version 4.4.1 (2024-06-14 ucrt)"
```